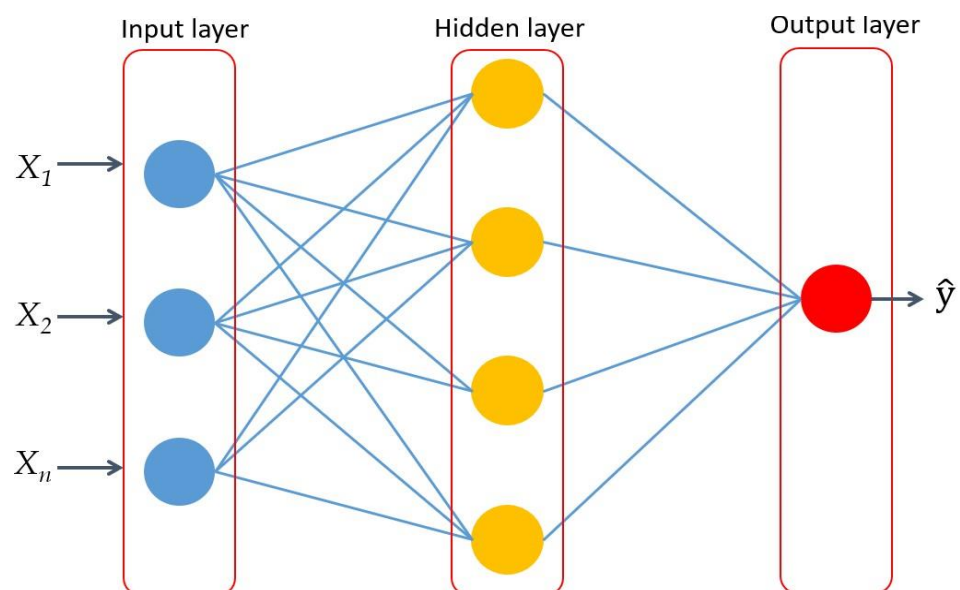


MEMBANGUN MODEL ARTIFICIAL NEURAL NETWORK (ANN)

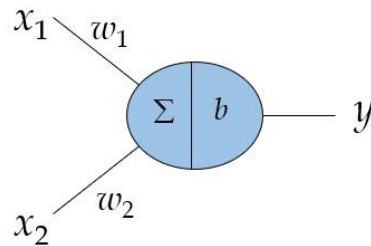
Artificial Neural Network (ANN) merupakan salah satu metode mesin pembelajaran yang terinspirasi oleh cara kerja jaringan saraf di otak manusia dalam menyelesaikan masalah melalui perubahan bobot sinapsisnya. ANN adalah metode *supervised learning* yang mampu mempelajari pola suatu data dengan menggunakan rekam data di masa lalu sebagai data latih. Pada ANN, terdapat neuron yang terhubung dengan neuron lainnya. Neuron-neuron tersebut akan mentransformasikan informasi yang diterima menuju ke neuron yang lain. Metode ANN merupakan arsitektur *Fully Connected Network* (FCN) dimana setiap hubungan antar neuron ini terdapat nilai bobot. Arsitektur FCN terdiri dari 3 layer, yaitu input layer, hidden layer, dan output layer. FCN dapat diklasifikasikan menjadi 2 (dua) jenis, yaitu single-layer dan multi-layer.

- Arsitektur single-layer terdiri dari 2 layer, yaitu input dan output. Input layer menerima masukan, sedangkan output layer memberikan respon berdasarkan masukan.
- Arsitektur multi-layer seperti pada Gambar 1 memiliki struktur tambahan yaitu hidden layer yang menghubungkan input layer dan output layer. Jumlah hidden layer merepresentasikan tingkat kompleksitas sebuah jaringan, semakin kompleks permasalahan maka akan dibutuhkan semakin banyak jumlah hidden unit.



Gambar 1. Arsitektur Multi-layer FCN

Jumlah *neuron* pada *input layer* merepresentasikan jumlah fitur pada sebuah dataset, jika input berupa data citra maka jumlah neuron sesuai ukuran citra input. Jumlah *neuron* pada *hidden layer* umumnya lebih banyak daripada jumlah neuron di *output layer*. Sedangkan jumlah *neuron* pada *output layer* sesuai dengan jumlah kelas pada permasalahan klasifikasi. Pada kasus prediksi dan klasifikasi dua kelas, jumlah *neuron* pada *output layer* adalah satu neuron.



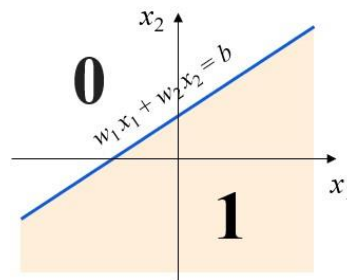
Gambar 2. Representasi sebuah neuron pada metode ANN

Gambar 2 menunjukkan bahwa setiap *neuron* mempunyai luaran y yang diperoleh dari persamaan linear yaitu penjumlahan dari perkalian input x dan bobot w dan menambahkan bias b . Perhitungan luaran pada setiap neuron dapat dilihat pada persamaan 1.

$$\hat{y} = a(\sum_{i=1}^m w_i x_i + b)$$

(1) Dimana a adalah fungsi aktivasi.

Sebuah neuron pada ANN dapat menyelesaikan permasalahan klasifikasi biner sebagai fungsi pemisah (*hyperspace separation*) dan *binary threshold* seperti ditunjukkan pada Gambar 3.



Gambar 3. Sebuah neuron dapat menyelesaikan permasalahan klasifikasi biner

Permasalahan klasifikasi dapat dikategorikan sebagai:

- Permasalahan Linear, misalnya fungsi OR dan AND
- Permasalahan Non-Linear, misalnya fungsi XOR

Pada ANN, fungsi aktivasi dapat merubah neuron menjadi non-linear. Beberapa contoh fungsi aktivasi yang umum digunakan pada metode ANN sebagai berikut: 1. Fungsi Sigmoid

$$a(z) = \frac{1}{1+e^{-z}} \quad (2)$$

2. Fungsi Gaussian

$$a(z) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(z-\mu)^2}{2\sigma^2}}$$

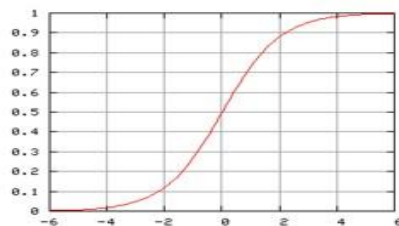
3. Fungsi Hyperbolic Tangent

$$a(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad (4)$$

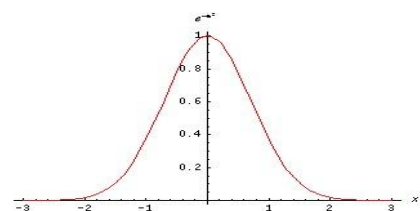
4. Fungsi Rectified Linear Unit (ReLU)

$$a(z) = \max(0, z) \quad (5)$$

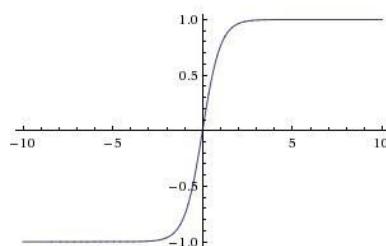
Adapun grafik dari fungsi aktivasi dapat dilihat pada Gambar 4.



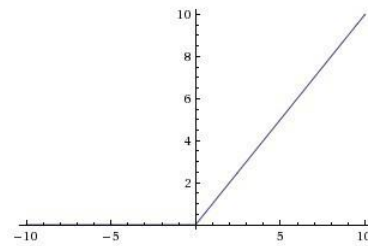
(a) Sigmoid



(b) Gaussian



(c) Hyperbolic Tangent



(d) ReLU

Gambar 4. Contoh grafik fungsi aktivasi

Pada permasalahan non-linear dan permasalahan yang lebih kompleks, dapat menggunakan arsitektur Multi-layer ANN.

Desain Arsitektur ANN

Terdiri dari tiga layer yaitu:

- input layer
- hidden layer
- output layer

Hubungan antar neuron pada ANN merupakan *Fully connected layer*. Jumlah *hidden layer* sebaiknya disesuaikan dengan kompleksitas permasalahan. Jumlah neuron pada *hidden layer* umumnya lebih banyak daripada jumlah *neuron* di *output layer*.

Penentuan jumlah *neuron* pada *input layer*

- Jumlah neuron sesuai dengan jumlah fitur pada data input

Penentuan jumlah *neuron* pada *output layer*

- Jumlah neuron sesuai dengan permasalahan

- Pada permasalahan klasifikasi biner dan regresi bisa menggunakan hanya satu *neuron*
- Pada permasalahan klasifikasi *multiclass* menggunakan jumlah *neuron* sesuai jumlah label kelasnya, misalnya: 10 neuron pada pengenalan angka

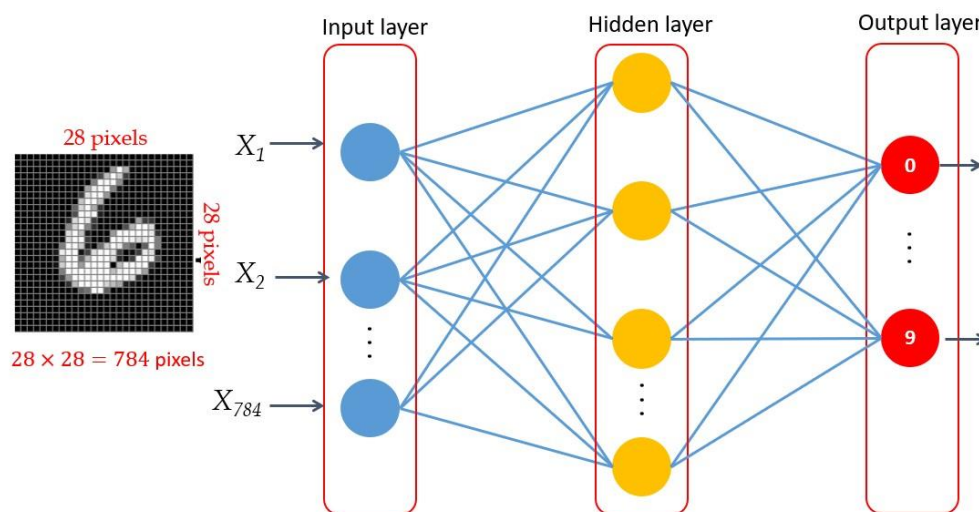
Penentuan jumlah *hidden layer*:

- Semakin banyak jumlah layer memerlukan komputasi waktu lebih lama
- Jumlah layer sebaiknya disesuaikan dengan kompleksitas permasalahan

Penentuan jumlah node (neuron) pada *hidden layer*:

- Semakin banyak jumlah node memungkinkan mempelajari pola yang lebih rumit
- Untuk mencegah overfitting sebaiknya menambah jumlah node secara bertahap

Contoh desain arsitektur Multi-layer ANN untuk pengenalan angka dapat dilihat pada Gambar 5.



Gambar 5. Contoh desain arsitektur ANN pada pengenalan angka

Proses pembelajaran pada metode ANN adalah proses mendapatkan nilai bobot yang optimal pada setiap koneksi antara neuron yang memenuhi kriteria fungsi obyektif. Fungsi obyektif metode ANN berupa fungsi *loss* seperti pada persamaan 6. Persamaan 7 merupakan fungsi *Means square loss* dan persamaan 8 merupakan fungsi *Cross entropy loss*. Fungsi *Means square loss* umumnya digunakan pada permasalahan regresi, sedangkan pada permasalahan klasifikasi menggunakan fungsi *Cross entropy loss*. Pada kasus permasalahan klasifikasi biner, umumnya menggunakan fungsi *Binary Cross entropy loss*.

(6)

$$J(W) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; W), y^{(i)})$$

$$J(W) = \frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - f(x^{(i)}; W) \right)^2 \quad (7)$$

$$J(W) = \frac{1}{n} \sum_{i=1}^n y^{(i)} \log \left(f(x^{(i)}; W) \right) + (1 - y^{(i)}) \log \left(1 - f(x^{(i)}; W) \right) \quad (8)$$

Algoritma Pembelajaran Backpropagation

Proses pembelajaran pada ANN bertujuan untuk mendapatkan bobot-bobot yang optimal melalui proses meminimalisasi fungsi obyektif. Algoritma *backpropagation* merupakan algoritma pembelajaran yang digunakan oleh metode ANN dalam proses mengupdate nilai bobot. Pembelajaran dengan algoritma *backpropagation* terdiri atas tiga tahap, yaitu (i) *feed forward*, (ii) propagasi *error*, dan (iii) pembaruan bobot dan bias.

Tahap satu adalah *feed forward* yaitu menghitung luaran dari semua neuron yang diaktifkan menggunakan suatu fungsi aktivasi. Pada tahap *feed forward* ini, persamaan 1 digunakan untuk estimasi nilai luaran setiap neuron. Sinyal luaran setiap neuron ini kemudian dikirim ke seluruh neuron pada layer selanjutnya hingga *output layer*.

Tahap kedua adalah menghitung *propagate error* antara hasil perhitungan nilai luaran pada output layer (nilai prediksi) dengan nilai target yang sudah ditentukan pada data latih. Tahap ketiga adalah mengupdate nilai bobot pada semua layer dengan fungsi tujuan adalah meminimalkan fungsi loss seperti pada persamaan 9 dan 10.

$$W^* = \underset{W}{\operatorname{argmin}} J(W) \quad (9)$$

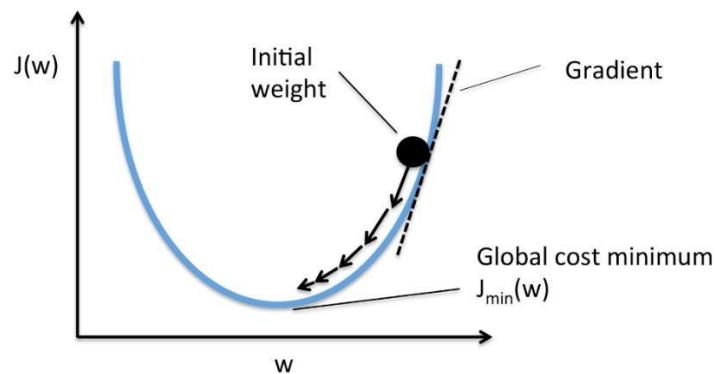
$$W^* = \underset{W}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; W), y^{(i)}) \quad (10)$$

Algoritma pembelajaran pada metode ANN umumnya menggunakan algoritma *backpropagation gradient descent*.

Algoritma Backpropagation Gradient descent

- Inisialisasi bobot secara random
- Iterasi sampai konvergen
- Hitung *gradient*, $\frac{\partial J(W)}{\partial W}$
- Update bobot, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$

- Mengembalikan nilai bobot



Gambar 6. Grafik optimasi nilai bobot dengan fungsi obyektif

Algoritma Stochastic Gradient Descent

- | | |
|--|--|
| <ul style="list-style-type: none"> • Inisialisasi <u>bobot secara random</u> • <u>Melakukan iterasi sampai konvergen atau maksimum iterasi</u> <ul style="list-style-type: none"> • Baca satu data poin i • Hitung gradient, $\frac{\partial J_i(w)}{\partial w}$ • Update bobot, $W \leftarrow W - \eta \frac{\partial J(w)}{\partial w}$ • <u>Mengembalikan nilai bobot</u> | <ul style="list-style-type: none"> • Inisialisasi <u>bobot secara random</u> • <u>Melakukan iterasi sampai konvergen atau maksimum iterasi</u> <ul style="list-style-type: none"> • Baca batch B data poin • Hitung gradient, $\frac{\partial J(W)}{\partial W} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(W)}{\partial W}$ • Update bobot, $W \leftarrow W - \eta \frac{\partial J(w)}{\partial w}$ • <u>Mengembalikan nilai bobot</u> |
|--|--|

Tahapan Pembelajaran Multi-layer Perceptron ANN

- Langkah 0 – Inisialisasi bobot, *learning rate*, maksimum iterasi
- Langkah 1 – Membaca vektor input X
- Langkah 2 – Lakukan iterasi (*epoch*)
- Langkah 3 – Hitung luaran neuron di hidden layer dan output layer
- Langkah 4 – Hitung *back propagate error* (pada output layer dan hidden layer)
- Langkah 5 – Perbarui semua bobot (pada output layer dan hidden layer)
- Langkah 6 – Ulangi langkah 3 – 5 hingga bobot konvergen atau maksimum iterasi
- Langkah 7 – Luaran berupa matrik bobot (pada output layer dan hidden layer)

Contoh perhitungan dari penerapan algoritma pembelajaran perceptron Multi-layer ANN pada kasus operasi logika AND dapat dilihat pada tabel dibawah ini:

Epoch	Inputs		Desired output Y_d	Initial weights		Actual output Y	Error e	Final weights	
	x_1	x_2		w_1	w_2			w_1	w_2
1	0	0	0	0.3	-0.1	0	0	0.3	-0.1
	0	1	0	0.3	-0.1	0	0	0.3	-0.1
	1	0	0	0.3	-0.1	1	-1	0.2	-0.1
	1	1	1	0.2	-0.1	0	1	0.3	0.0
2	0	0	0	0.3	0.0	0	0	0.3	0.0
	0	1	0	0.3	0.0	0	0	0.3	0.0
	1	0	0	0.3	0.0	1	-1	0.2	0.0
	1	1	1	0.2	0.0	1	0	0.2	0.0
3	0	0	0	0.2	0.0	0	0	0.2	0.0
	0	1	0	0.2	0.0	0	0	0.2	0.0
	1	0	0	0.2	0.0	1	-1	0.1	0.0
	1	1	1	0.1	0.0	0	1	0.2	0.1
4	0	0	0	0.2	0.1	0	0	0.2	0.1
	0	1	0	0.2	0.1	0	0	0.2	0.1
	1	0	0	0.2	0.1	1	-1	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1
5	0	0	0	0.1	0.1	0	0	0.1	0.1
	0	1	0	0.1	0.1	0	0	0.1	0.1
	1	0	0	0.1	0.1	0	0	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1

Threshold: $\theta = 0.2$; learning rate: $\alpha = 0.1$.

Sumber: MICHAEL NEGNEVITSKY, Artificial Intelligence: A Guide to Intelligent Systems, Second Edition, Addison Wesley, 2005

Strategi Pembelajaran Metode ANN

Ada beberapa strategi pembelajaran pada metode ANN untuk menangani beberapa kendala dalam proses pelatihan yaitu:

1. Permasalahan *overfitting*

Overfitting adalah permasalahan yang umum dihadapi pada saat membangun model ANN yaitu kinerja dari data latih lebih tinggi daripada kinerja dari data validasi. Strategi yang bisa dilakukan untuk mencegah *overfitting* adalah:

- regularisasi
- melakukan augmentasi data
- penggunaan *dropout* pada saat proses pelatihan
- Early stopping

2. Permasalahan kinerja model yang rendah

Pada proses pembangunan model terkadang kinerja model masih rendah, sehingga model tersebut belum bisa digunakan untuk proses prediksi pada data uji. Strategi untuk meningkatkan kinerja model yang masih rendah sebagai berikut:

- Modifikasi *network*
- Merubah fungsi aktivasi
- Menggunakan *adaptive learning rate*

Regularization

Regularisasi dilakukan untuk mengurangi generalization error dengan mencegah model lebih kompleks. Penerapan regularisasi dengan cara menambahkan regularization term pada semua parameter (bobot) ke fungsi obyektif.

- Regularization L1 norm ○ Menambahkan sum of the absolute weights sebagai penalty term ke fungsi obyektif
- Regularization L2 norm (weight decay) ○ Menambahkan sum of the squared weights sebagai penalty term ke fungsi obyektif

Augmentasi Data

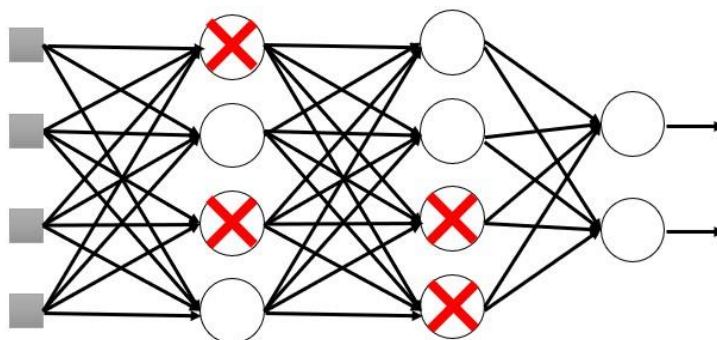
Augmentasi data adalah proses memperbanyak variasi data latih, sehingga model yang dihasilkan lebih baik dalam memprediksi data uji. Metode augmentasi data yang digunakan tergantung dari jenis data input. Pada data citra, metode augmentasi yang umum digunakan adalah rotasi, translasi, *flip*, dan *zoom* (dapat dilihat pada Gambar 7).



Gambar 7. Ilustrasi penggunaan Augmentasi Data

Konsep Dropout

Dropout adalah cara untuk meregulasi parameter untuk menghindari *overfitting* sehingga model lebih general. Umumnya dilakukan pada *Fully Connected Layer*. Selama proses training, setiap iterasi sebelum menghitung gradient dilakukan penentuan neuron yang diset tidak aktif sesuai prosentase dropout $p\%$. Contoh ilustrasi konsep Dropout pada saat proses pelatihan dapat dilihat pada Gambar 8.



Gambar 8. Ilustrasi penggunaan *Dropout*

Konsep *Early Stopping*

Iterasi pada saat training dihentikan jika generalization error mulai naik seperti pada Gambar 9.



Gambar 9. Ilustrasi penggunaan *Early Stopping*

Adaptive Learning Rate

Nilai *learning rate* berpengaruh pada perhitungan bobot baru, umumnya penggunaan *learning rate* yang menyesuaikan nilai gradien (*adaptive learning rate*) menunjukkan kinerja model yang lebih baik. Contoh algoritma *adaptive learning rate*:

- SGD [Kiefer & Wolfowitz, Ann. Math. Statist. 1952]
- Adagrad [John Duchi, JMLR 2011]
- Adadelata [Matthew D. Zeiler, arXiv 2012]
- AdaSecant [Caglar Gulcehre, arXiv 2014]
- Adam [Diederik P. Kingma, ICLR 2015]
- RMSProp <https://www.youtube.com/watch?v=O3sxAc4hxZU>

Tahapan Implementasi ANN

1. Load data: membaca file data input
 2. Split data: membagi data menjadi data latih, data validasi, data uji
 3. Define model: merancang arsitektur atau model ANN
 4. Compile model: menjalankan model ANN yang sudah dirancang
 5. Fit model: membangun model ANN berdasarkan data latih
 6. Evaluation model: mengevaluasi model ANN berdasarkan data validasi
 7. Prediction: memprediksi output dari data uji menggunakan model ANN yang terbaik
- TOOLS: Scikit-learn**

sklearn.neural_network.MLPClassifier

```
class sklearn.neural_network.MLPClassifier(hidden_layer_sizes=100, activation='relu', *, solver='adam', alpha=0.0001,
batch_size='auto', learning_rate='constant', learning_rate_init=0.001, power_t=0.5, max_iter=200, shuffle=True,
random_state=None, tol=0.0001, verbose=False, warm_start=False, momentum=0.9, nesterovs_momentum=True,
early_stopping=False, validation_fraction=0.1, beta_1=0.9, beta_2=0.999, epsilon=1e-08, n_iter_no_change=10, max_fun=15000)
```

[source]

Parameter MLPClassifier:

- hidden_layer size: jumlah neuron di *hidden layer*
- activation: fungsi aktivasi yang digunakan di *hidden layer*
- solver: metode adaptive learning rate yang digunakan
- batch_size: ukuran batch
- learning_rate_init: inisialisasi *learning rate*
- max_iter: *maksimum iterasi*
- early_stopping: bernilai false jika tidak menerapkan *early stopping*

Contoh implementasi ANN pada dataset Iris menggunakan library

Scikit-learn 1. Load Data

sklearn.datasets.load_iris

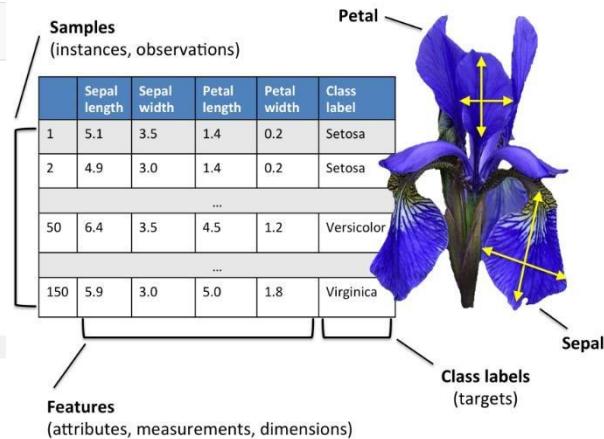
```
sklearn.datasets.load_iris(*, return_X_y=False, as_frame=False)
```

Load and return the iris dataset (classification).

The iris dataset is a classic and very easy multi-class classification dataset.

Classes	3
Samples per class	50
Samples total	150
Dimensionality	4
Features	real, positive

4



```
from sklearn import datasets
```

```
iris = datasets.load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

2. Split Data

`sklearn.model_selection.train_test_split`

```
sklearn.model_selection.train_test_split(*arrays, test_size=None, train_size=None, random_state=None, shuffle=True, stratify=None)
```

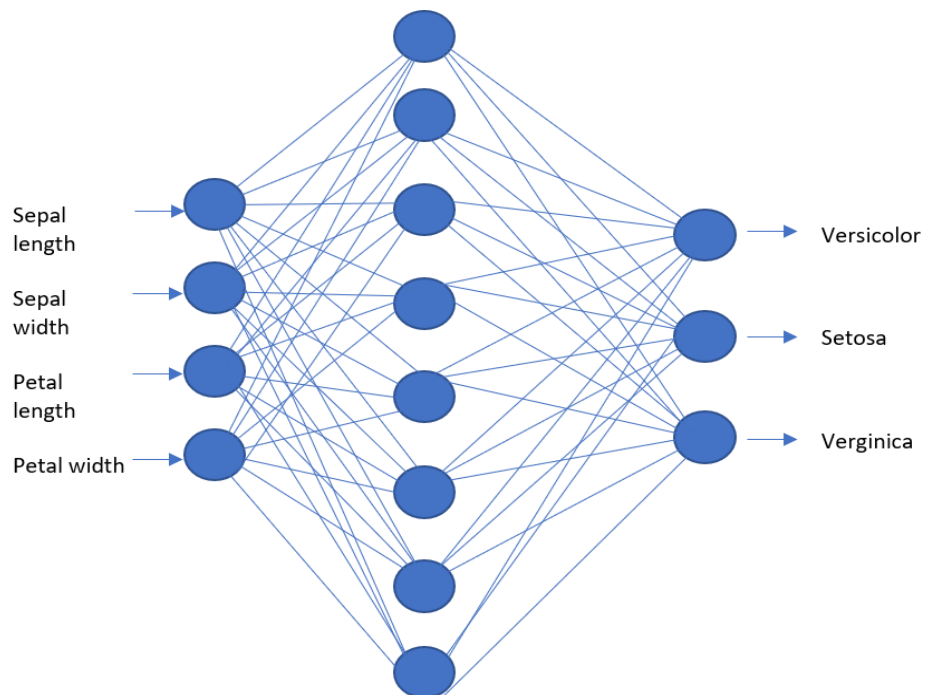
[\[source\]](#)

```
from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(X, y,
test_size=.10) X_train, X_val, Y_train, Y_val =
train_test_split(X_train, Y_train, test_size=.15) print('X_train',
X_train.shape)

print('X_val', X_val.shape)
print('X_test', X_test.shape)
```

3. Define Model



Iris Versicolor



Iris Setosa



Iris Virginica

```
from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(hidden_layer_sizes=(64, ), activation='relu',
learning_rate_init=0.001, max_iter=100)
```

4. Fit Model dan Evaluation Model

```
from sklearn.metrics import accuracy_score
mlp.fit(X_train,
Y_train)
prediksi_val = mlp.predict(X_val)
acc_val = accuracy_score(Y_val, prediksi_val)
print('Akurasi Validasi Training ANN:', acc_val)
```

5. Prediction

```
from sklearn.metrics import accuracy_score, plot_confusion_matrix
prediksi_test = mlp.predict(X_test) acc_test =
accuracy_score(Y_test, prediksi_test)
print('Akurasi Testing ANN:', acc_test)
plot_confusion_matrix(mlp, X_test, Y_test)
```

TOOLS: TENSOR FLOW DAN KERAS

- TensorFlow merupakan *end-to-end open-source platform* untuk mesin pembelajaran (<https://www.tensorflow.org/overview/>)
- Keras merupakan sebuah deep learning API berbasis Python, berjalan diatas platform TensorFlow. Dimana fokus pengembangan adalah untuk mempercepat proses eksperimen. (https://keras.io/getting_started/)



Keras Model Sequential dan Layers

Pembangunan model pada Keras dapat dilihat pada link berikut ini:

<https://keras.io/api/models/> dan https://keras.io/guides/sequential_model/

Contoh pembangunan model Sequential pada Keras seperti berikut ini:

```
model = keras.Sequential(
    [
        layers.Dense(64, activation="relu", name="layer1"),
        layers.Dense(32, activation="relu", name="layer2"),
        layers.Dense(4, name="layer3"),
    ]
)
```

atau

```
model = keras.Sequential()
model.add(layers.Dense(64, activation="relu"))
model.add(layers.Dense(32, activation="relu"))
model.add(layers.Dense(4))
```

Contoh implementasi ANN pada dataset Iris menggunakan library Keras

1. Load Data dan Split Data

```
from sklearn import datasets
from sklearn.model_selection import train_test_split

from keras.utils import to_categorical
iris = datasets.load_iris()

X = iris.data
y = iris.target

X_train, X_test, Y_train, Y_test = train_test_split(X, y, test_size=.10)
X_train, X_val, Y_train, Y_val = train_test_split(X_train, Y_train, test_size=.2)
print('X_train', X_train.shape)
print('X_val', X_val.shape)
print('X_test', X_test.shape)
Y_train = to_categorical(Y_train, 3)
Y_val = to_categorical(Y_val, 3)
Y_test = to_categorical(Y_test, 3)
```

2. Define Model dan Compile Model

```
from keras.models import Sequential
from keras.layers import Flatten, Dense

model = Sequential()
model.add(Flatten())
model.add(Dense(64, activation='relu'))
model.add(Dense(3, activation='softmax'))
model.compile(optimizer='adam', loss='categorical_crossentropy',
              metrics=['acc'])
```

3. Fit Model dan Evaluation Model

```
model.fit(X_train,Y_train,epochs=64,batch_size=5,validation_data=(X_test,Y_test))
model.summary()
loss, accuracy = model.evaluate(X_test, Y_test)
print('Akurasi Testing MLP:', accuracy)
```

Model: "sequential"

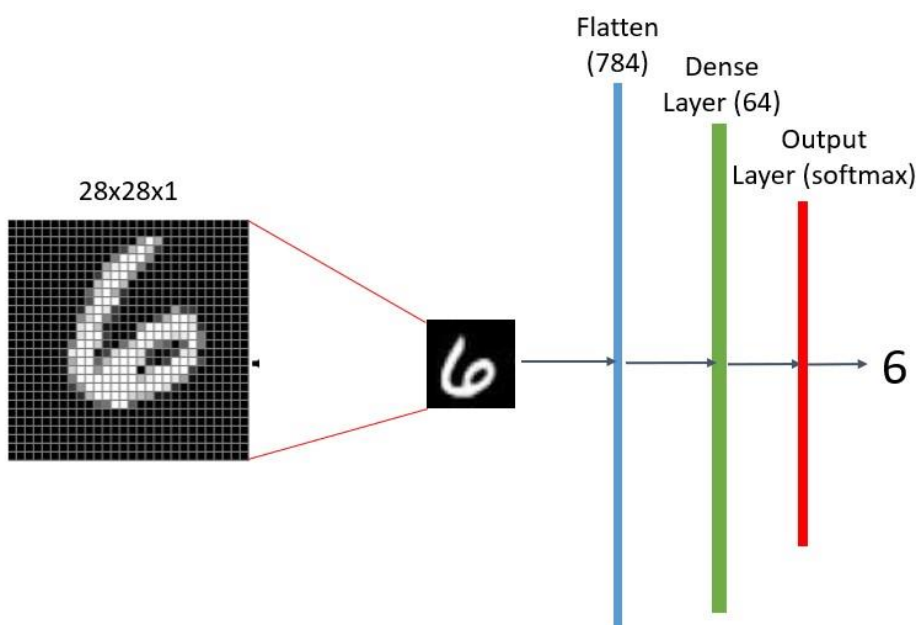
Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 4)	0
dense (Dense)	(None, 64)	320
dense_1 (Dense)	(None, 3)	195

=====
Total params: 515
Trainable params: 515
Non-trainable params: 0

1/1 [=====] - 0s 0s/step - loss: 0.0393 - acc: 1.0000
Akurasi Testing ANN: 1.0

Contoh Penerapan Arsitektur ANN Pada Permasalahan Pengenalan Angka

Arsitektur MLP ANN yang akan dibangun untuk pengenalan angka dari citra tulisan tangan seperti pada Gambar 10. Arsitektur MLP yang digunakan adalah *input layer* dengan jumlah neuron sesuai ukuran citra input, jumlah neuron pada *hidden layer* adalah 64, dan jumlah neuron pada *output layer* adalah 10 sesuai jumlah angka yang akan diklasifikasi. Dataset yang digunakan adalah dataset MNITS Handwritten Digit.



Gambar 10. Contoh arsitektur Multi-layer ANN untuk pengenalan angka

Dataset MNIST Handwritten Digit dibagi menjadi 3 bagian:

- 55,000 training data
- 10,000 test data
- 5,000 validation data

Setiap citra berukuran 28×28 *pixels* dan label kelas berupa *one hot encoded*. Dibawah ini adalah contoh citra pada dataset MNIST beserta contoh label kelasnya.

label = 5 	label = 0 	label = 4 	label = 1 	label = 9 	0	[1 0 0 0 0 0 0 0 0 0]
label = 2 	label = 1 	label = 3 	label = 1 	label = 4 	1	[0 1 0 0 0 0 0 0 0 0]
label = 2 	label = 1 	label = 3 	label = 1 	label = 4 	2	[0 0 1 0 0 0 0 0 0 0]
label = 3 	label = 5 	label = 3 	label = 6 	label = 1 	3	[0 0 0 1 0 0 0 0 0 0]
label = 3 	label = 5 	label = 3 	label = 6 	label = 1 	4	[0 0 0 0 1 0 0 0 0 0]
label = 7 	label = 2 	label = 8 	label = 6 	label = 9 	5	[0 0 0 0 0 1 0 0 0 0]
					6	[0 0 0 0 0 0 1 0 0 0]
					7	[0 0 0 0 0 0 0 1 0 0]
					8	[0 0 0 0 0 0 0 0 1 0]

1. Load Data

```
import keras from keras.datasets
import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train.reshape(-1, 28,28,1)
X_test = X_test.reshape(-1, 28,28,1)
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
X_train /= 255 X_test /= 255 y_train =
keras.utils.to_categorical(y_train, 10) y_test =
keras.utils.to_categorical(y_test, 10)
```

2. Define Model Multi-layer ANN

```
from keras.models import Sequential from
keras.layers import Flatten, Dense

model1 = Sequential()

model1.add(Flatten())
```

```
model1.add(Dense(64,activation='relu'))
model1.add(Dense(10,activation='softmax'))
```

```
model1.summary()
```

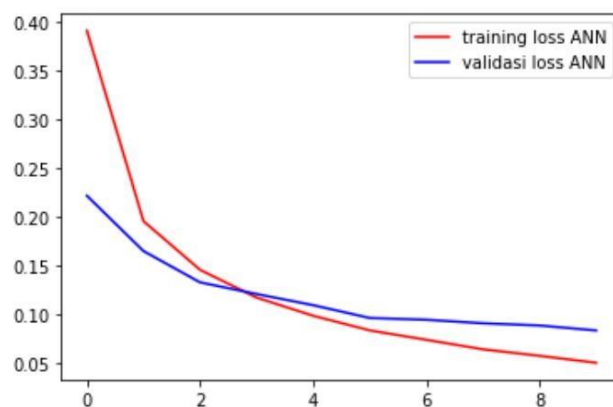
Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 64)	50240
dense_1 (Dense)	(None, 10)	650
Total params: 50,890		
Trainable params: 50,890		
Non-trainable params: 0		

3. Model, Fit Model, Save Model, dan Evaluasi Model

```
model1.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['acc']) history =
model1.fit(X_train,y_train,epochs=10,batch_size=100,validation_data=(X_test,y_test)) model1.save('my_model1.h5') model1.evaluate(X_test,y_test)
```

Visualiasasi Evaluasi Model



4. Load Model dan Prediction

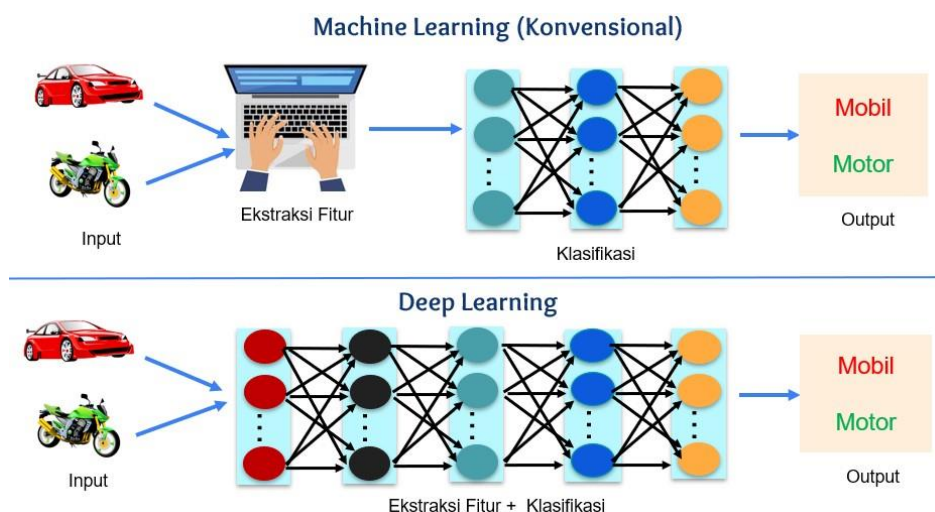
```
import numpy as np from keras.models import
load_model model_simpan = load_model('my_model.h5')
pred = model_simpan.predict(X_test) print('label
actual:',np.argmax(y_test[30])) print('label
prediction:',np.argmax(pred[30]))
```

Pengantar Deep Learning

Pendekatan klasifikasi secara konvensional umumnya melakukan ekstraksi fitur secara terpisah kemudian dilanjutkan proses pembelajaran menggunakan metode klasifikasi konvensional (Gambar 11). Kelemahan pendekatan konvensional:

- Memerlukan waktu dan pengetahuan lebih untuk ekstraksi fitur
- Sangat tergantung pada satu domain permasalahan saja sehingga tidak berlaku general

Pendekatan klasifikasi berbasis Deep learning mempelajari representasi hirarki (pola fitur) secara otomatis melalui beberapa tahapan proses *feature learning*.

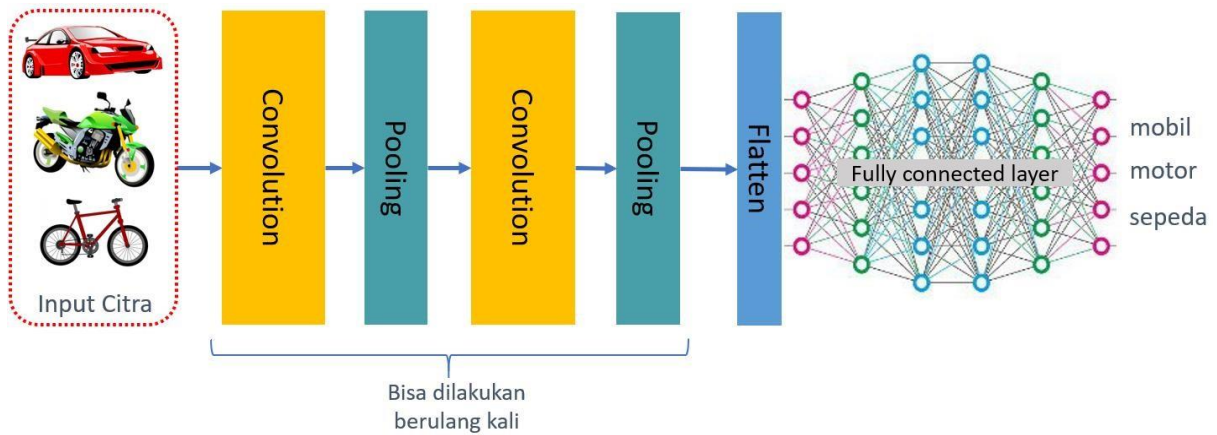


Gambar 11. Ilustrasi konsep machine learning dan deep learning

CONVOLUTIONAL NEURAL NETWORK

Convolutional Neural Network (CNN) adalah salah satu jenis arsitektur dari ANN untuk memproses data yang memiliki susunan topologi seperti grid (I. Goodfellow, dkk., 2016). Misalnya data *time-series* merupakan data yang berbentuk grid 1D, yang menyatakan sampel data terhadap satuan waktu. Contoh lain data citra merupakan data grid 2D yang terdiri dari nilai piksel. Penamaan *Convolutional* karena penggunaan operator konvolusi pada salah satu layer ANN yang menggantikan operasi aljabar matriks biasa. Tahapan

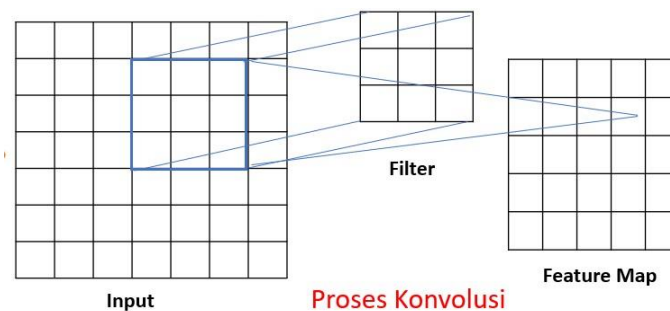
pada layer CNN umumnya terbagi menjadi 3 tahap, yaitu: 1) convolution, 2) pooling 3) flatten 4) fully connected layer. Arsitektur CNN untuk proses klasifikasi diilustrasikan pada Gambar 12.



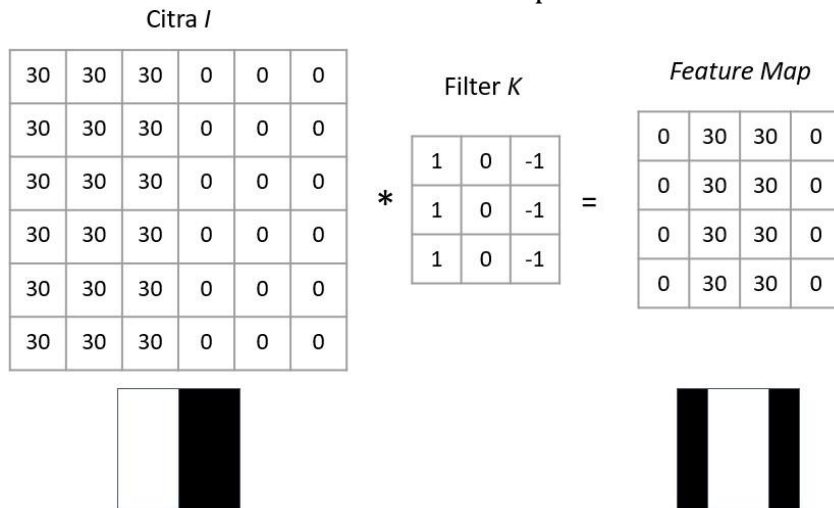
Gambar 12. Arsitektur *Convolutional Neural Network* (CNN)

Tahap pertama merupakan tahap *convolution*, ilustrasi proses konvolusi dapat dilihat pada Gambar 13. Pada tahap ini, melakukan operasi konvolusi pada data input dengan kernel. Kernel pada CNN merupakan *array multidimensional* yang menjadi operator untuk melakukan operasi tertentu dengan array input seperti pada persamaan 11. Nilai yang terdapat pada kernel merupakan nilai yang diperoleh dari hasil learning. Hasil operasi aljabar antara input dengan kernel disebut dengan *feature map*. Proses operasi antara input dengan kernel ini disebut dengan proses ekstraksi fitur ke dalam *feature map*. Contoh proses konvolusi pada sebuah citra dapat dilihat pada Gambar 14.

$$(I * K)(i, j) == \sum_m \sum_n I(m, n) K(i + m, j + n) \quad (11)$$

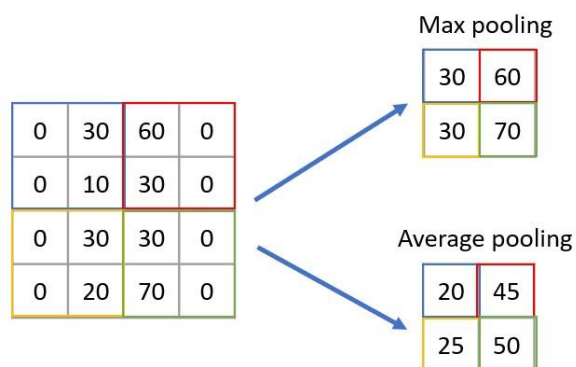


Gambar 13. Ilustrasi proses Konvolusi



Gambar 14. Contoh proses Konvolusi pada input citra

Tahap kedua merupakan tahap *detector* yaitu hasil dari tahap sebelumnya dijadikan sebagai input ke dalam suatu fungsi aktivasi misalnya *Rectified linear unit* (ReLU). Tahap ketiga merupakan tahap pooling yaitu mereduksi dimensi dari *input array* yang masuk dengan memanfaatkan fungsi maksimum, penjumlahan, atau rata-rata, yang disebut *maxpooling*, *sum-pooling*, atau *average-pooling*. Luaran dari tahapan ini menghasilkan dimensi data menjadi lebih kecil namun tanpa menghilangkan informasi penting dari data yang diterima. *Pooling layer* yang umum digunakan untuk mengurangi ukuran gambar menjadi lebih kecil (*downsample*) dan mengekstrak *salient features* adalah *Maximum pooling* dan *Average pooling*. Ilustrasi konsep dari *Max-pooling* dan *Averagepooling* dapat dilihat pada Gambar 15.

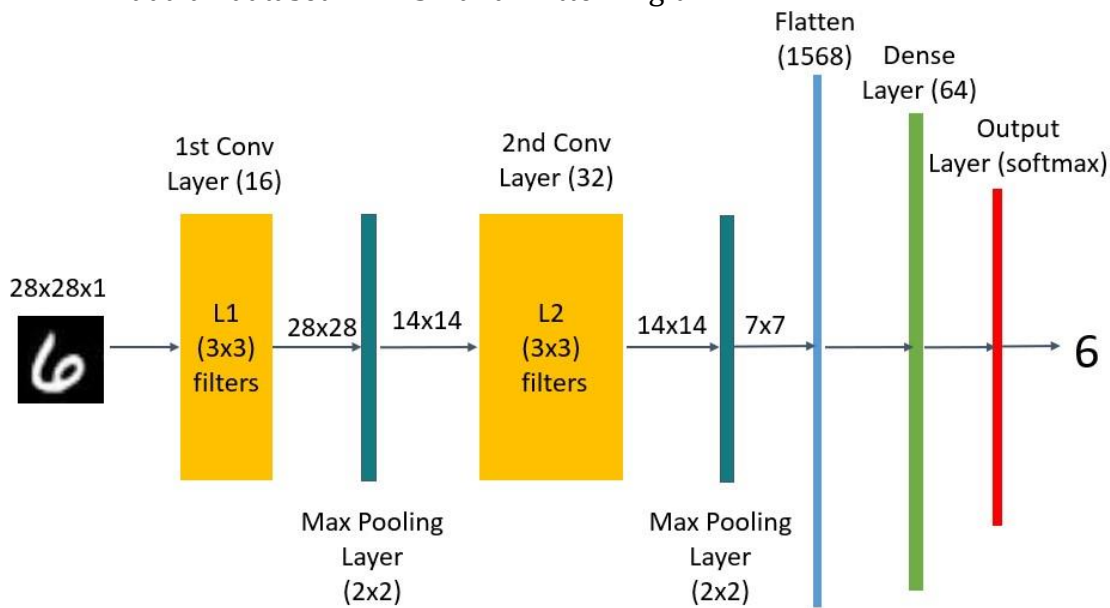


Gambar 15. Contoh proses Pooling pada sebuah *feature map*

Setelah menggunakan beberapa *convolutional* dan *pooling layer*, umumnya untuk permasalahan klasifikasi digunakan layer tambahan berupa *fully connected layer* sebagai tahap 4 yang melakukan proses *flattening* dari matriks menjadi sebuah vektor, dan selanjutnya digunakan untuk melakukan klasifikasi.

Contoh Penerapan Arsitektur CNN Pada Permasalahan Klasifikasi

Arsitektur CNN yang akan dibangun untuk pengenalan angka dari citra tulisan tangan seperti pada Gambar 16. Arsitektur CNN yang digunakan adalah *input layer* dengan jumlah neuron sesuai ukuran citra input, dua *convolution layer* dengan ukuran filter 3x3 sejumlah 16 dan 32, dua *max-pooling layer* dengan filter 2x2, sedangkan pada arsitektur FCN menggunakan 64 neuron pada *hidden layer* dan 10 neuron pada *output layer* sesuai jumlah angka yang akan diklasifikasi. Dataset yang digunakan adalah dataset MNITS Handwritten Digit.



Gambar 16. Contoh Arsitektur CNN untuk pengenalan angka

1. Define Model, Fit Model dan Evaluasi Model CNN

```
from keras.models import Sequential from keras.layers import
Conv2D, MaxPooling2D, Flatten, Dense
model2 = Sequential()
model2.add(Conv2D(16, (3,3), activation='relu', input_shape=(28,28,1), padding='s
ame')) model2.add(MaxPooling2D(2,2))
model2.add(Conv2D(32, (3,3), activation='relu', padding='same'))
model2.add(MaxPooling2D(2,2)) model2.add(Flatten())
model2.add(Dense(64, activation='relu'))
model2.add(Dense(10, activation='softmax'))
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 16)	160
max_pooling2d (MaxPooling2D)	(None, 14, 14, 16)	0
conv2d_1 (Conv2D)	(None, 14, 14, 32)	4640
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 32)	0
flatten_1 (Flatten)	(None, 1568)	0
dense_2 (Dense)	(None, 64)	100416
dense_3 (Dense)	(None, 10)	650

Total params: 105,866

Trainable params: 105,866

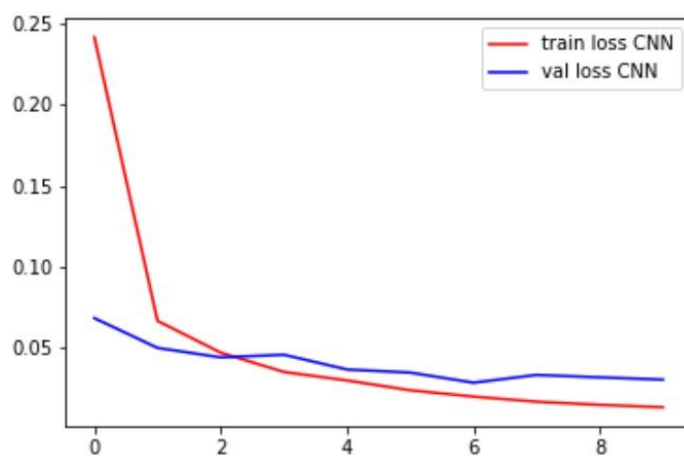
Non-trainable params: 0

2. Compile Model, Fit Model, Save Model, dan Evaluasi Model CNN

```
model.
```

```
Compile(optimizer='adam',loss='categorical_crossentropy',metrics=['acc'])  
history =
```

```
model.fit(X_train,y_train,epochs=10,batch_size=100,validation_data=(X_test,y_test))  
model.save('my_model.h5')
```



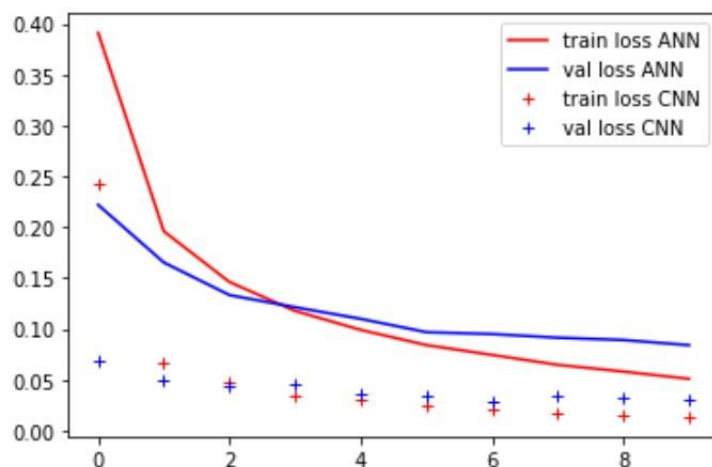
3. Load Model dan Prediction

```
import numpy as np from
keras.models import load_model
model_simpan2 = load_model('my_model2.h5')
pred = model_simpan2.predict(X_test)
print('label actual:',np.argmax(y_test[30]))
print('label prediction:',np.argmax(pred[30]))
```

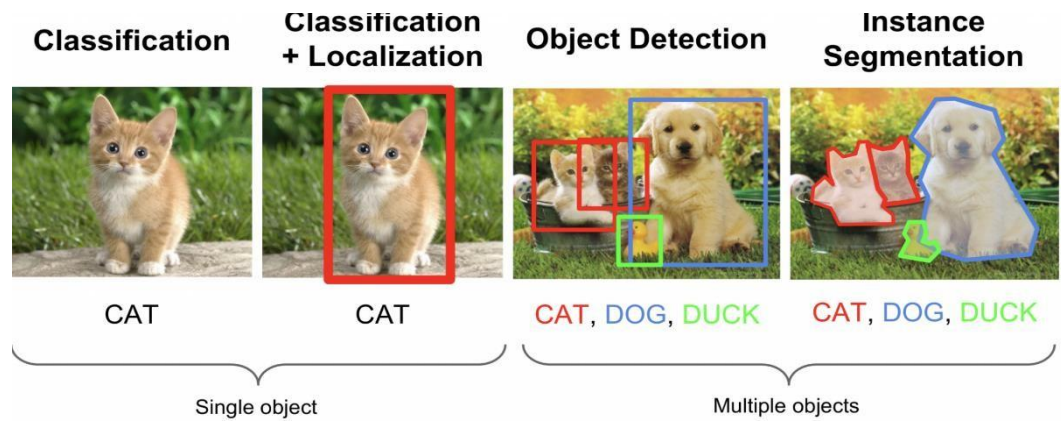
```
import matplotlib.pyplot as plt
```

```
epochs = range(10) loss1 = history1.history['loss'] val_loss1 =
history1.history['val_loss'] plt.plot(epochs,loss1,'r',label='train loss
ANN') plt.plot(epochs,val_loss1,'b',label='val loss ANN')
```

```
loss2 = history2.history['loss'] val_loss2 =
history2.history['val_loss'] plt.plot(epochs,loss2,'r+',label='train
loss CNN') plt.plot(epochs,val_loss2,'b+',label='val loss CNN')
plt.legend()
```



Tipe aplikasi yang menggunakan metode CNN misalnya *image classification*, *object detection*, dan *instance semantic segmentation* seperti pada Gambar 17.



Gambar 17. Contoh tipe aplikasi CNN

TIPE APLIKASI DAN VARIAN ARSITEKTUR CNN

Tipe Aplikasi	Arsitektur CNN
Image Classification	• LeNet-5 (1998) • AlexNet (2012) • GoogLeNet/Inception (2014) • VGGNet (2014) • ResNet (2015)
Object Detection	• R-CNN (2013) • Fast R-CNN (2014) • Faster R-CNN (2015) • Single Shot Detector (SSD) (2016) • YOLO (2016)
Semantic (Instance) Segmentation	• Fully Convolutional Network (FCN) (2015) • U-Net (2015) • Feature Pyramid Network (FPN) (2016) • Mask R-CNN (2017) • DeepLab (2016), DeepLabv3 (2017), DeepLabv3+ (2018)
Generative model	• Autoencoders, Variational Autoencoders (VAE) • Generative Adversarial Network (GAN)

RECURRENT NEURAL NETWORK

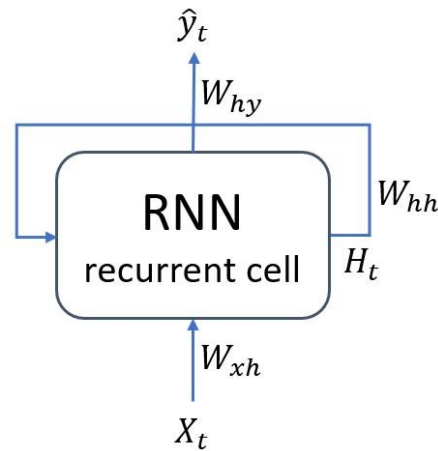
Recurrent Neural Network (RNN) adalah salah satu arsitektur ANN yang mampu merepresentasikan data sequential, misalnya teks, suara, time series, dan sebagainya. RNN berfokus pada sifat data yang mana data sebelumnya mempengaruhi data selanjutnya. Misalnya, diberikan sebuah sequence input $x = (x_1, x_2, x_3, \dots, x_t)$. Data x_t dipengaruhi oleh data

sebelumnya, ditulis sebagai $P(x_t | (x_1, x_2, x_3.. x_{t-1}))$. Ide utamanya adalah memorisasi yaitu mengingat keseluruhan sekuens. Sel RNN dapat diilustrasikan pada Gambar 18. Perhitungan hidden state H pada waktu ke- t bergantung pada fungsi aktivasi ϕ dari input pada waktu ke- t (X_t), hidden state sebelumnya (H_{t-1}) dan parameter pada ANN bias b . Konsep ini sesuai dengan prinsip recurrent yaitu mengingat kejadian sebelumnya. Persamaan RNN dapat dilihat pada persamaan 12 dan 13.

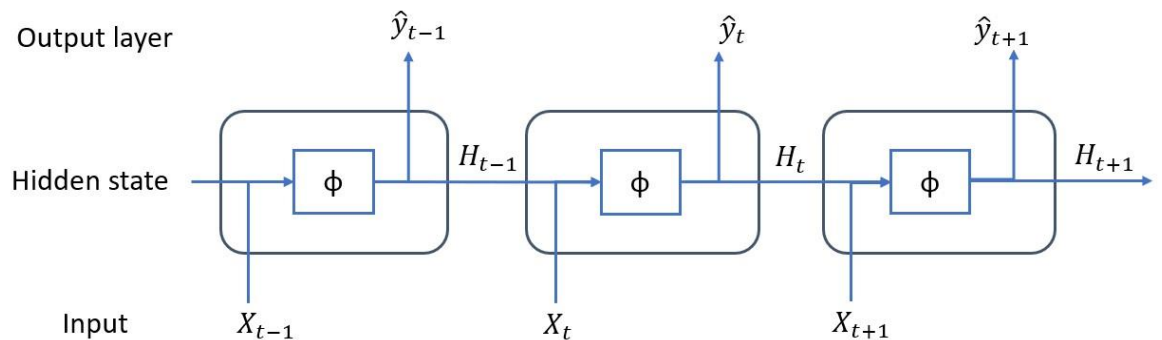
$$\hat{y}_t = H_t W_{hy} + b_y \quad (12)$$

$$H_t = \phi(X_t W_{xh} + H_{t-1} W_{hh} + b_h) \quad (13)$$

Gambar 19 menunjukkan urutan komputasi RNN pada tiga waktu yang berdekatan. Dalam tahap waktu t , perhitungan *hidden state* dapat diperlakukan sebagai entri dari lapisan yang terhubung penuh dengan fungsi aktivasi f setelah menggabungkan input X_t dengan *hidden state* H_{t-1} dari waktu sebelumnya. Output dari lapisan yang terhubung penuh adalah *hidden state* saat ini H_t . Parameter modelnya adalah gabungan dari W_{xh} dan W_{hh} , dengan bias b_h . *Hidden state* dari waktu saat ini t (H_t) akan berpartisipasi dalam menghitung *hidden state* H_{t+1} dari langkah waktu berikutnya $t+1$, yang hasilnya akan menjadi input untuk layer output yang terhubung penuh pada waktu saat ini.



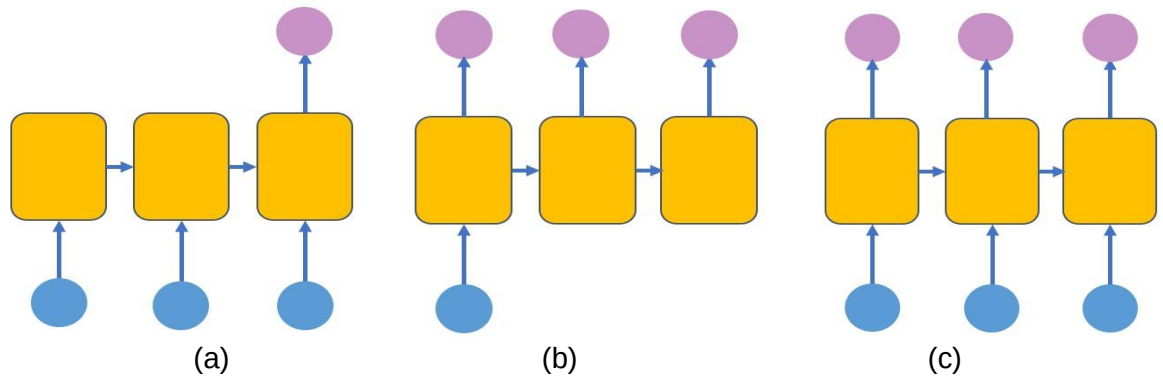
Gambar 18. Arsitektur *Recurrent Neural Network* (RNN)



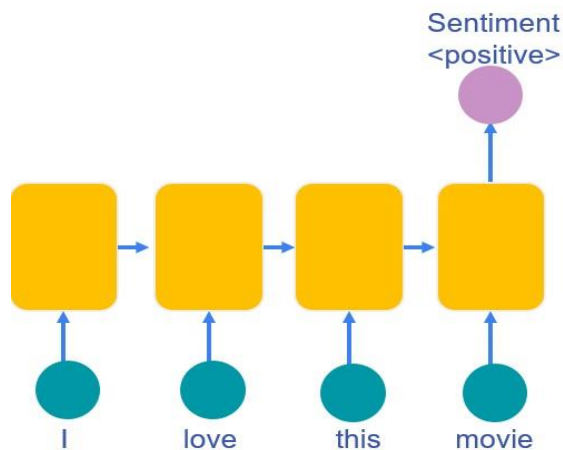
Gambar 19. Arsitektur *Recurrent Neural Network* (RNN)

Gambar 20 menunjukkan tipe-tipe arsitektur RNN dan Gambar 21 merupakan contoh penerapan salah satu tipe RNN pada aplikasi analisis sentiment.

- Contoh-contoh aplikasi dari tipe arsitektur RNN sebagai berikut:
- Many to One: Sentiment classification, Opinion mining, Speech recognition, Automated answer scoring, dsb.*
 - One to Many: Text generation, Image captioning, dsb.*
 - Many to Many: Translation, Forecasting, Music generation, Chatbot, dsb.*



Gambar 20. Tipe arsitektur RNN (a) *Many to One* (b) *One to Many* (c) *Many to Many*



Gambar 21. Contoh aplikasi analisis sentiment menggunakan arsitektur RNN

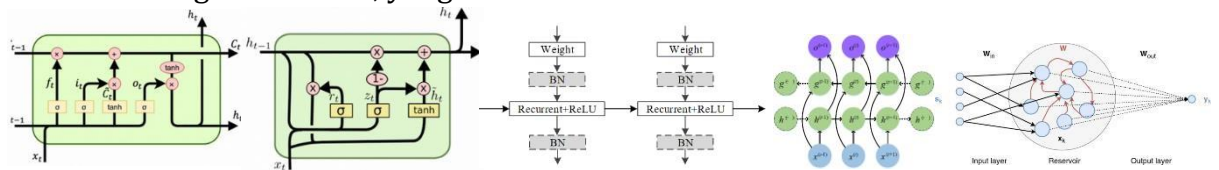
Varian Arsitektur RNN

Varian dari arsitektur RNN dapat dilihat pada Gambar 22 dan penjelasan setiap arsitektur sebagai berikut:

- **Long Short-Term Memory (LSTM)**
Merupakan salah satu jenis arsitektur RNN yang terdiri dari beberapa unit yaitu input gate, output gate, dan forget gate. LSTM diusulkan untuk mengatasi permasalahan *vanishing gradient* pada arsitektur RNN.
- **Gate Recurrent Unit (GRU)**

Merupakan simplifikasi dari arsitektur LSTM dengan menggabungkan input gate dan forget gate sehingga jumlah parameter lebih sedikit.

- **Independently RNN (IndRNN)**
Arsitektur RNN dimana setiap neuron dalam satu layer independen dari yang lain
- **Bi-directional RNN**
Merupakan arsitektur RNN menghubungkan dua *hidden layer* dari arah yang berlawanan ke output yang sama.
- **Echo State Network (ESN)**
Ide dasar ESN adalah untuk membuat jaringan berulang yang terhubung secara acak, yang disebut reservoir.



Gambar 22. Varian arsitektur RNN (LSTM, GRU, IndRNN, Bi-directional RNN, ESN)

REFERENSI

1. 虞台文, Feed-Forward Neural Networks, Course slides presentation
2. Andrew Ng, Machine Learning, Course slides presentation
3. Michael Negnevitsky, Artificial Intelligence: A Guide to Intelligent Systems, Second Edition, Addison Wesley, 2005.
4. Hung-yi Lee, Deep Learning Tutorial
5. Alexander Amini, Intro to Deep Learning, MIT 6.S191, 2021